Simple X-ray System

Semester:	Intelligent Technology
Project:	Simple X-ray system
Date:	8 Februari 2025
Authors:	Lake Lakeman

Contents

1. Introduction . . . 3
2. Project structure . . . 5
3. PatientAdministration Requirements . . . 6
4. CentralAcquisition Requirements . . . 8
   4.1. Hardware requirements . . . 8
   4.2. Software Requirements . . . 8
5. Geometry Requirements . . . 10
6. X-ray Generator Requirements . . . 13
   6.1. Hardware requirements . . . 13
   6.2. Software requirements . . . 14
7. Patient Administration ADT Details . . . 17
8. API . . . 18
9. Protocol requirements . . . 19
10. Sprint Content . . . 20
   10.1. Sprint 1 . . . 20
   10.2. Sprint 2 . . . 21
   10.3. Sprint 3 . . . 21
   10.4. Sprint 4 . . . 22

Version history

Version	Date	Author	Changes
1.0	2024-02-08	Lake Lakeman	Initial version
1.1	2024-03-11	Jaap Geurts	• Added electrical requirements • Added register drawings. • General improvements.
1.2	2024-03-21	Jaap Geurts	• Added project structure description
1.3	2024-02-03	Lake Lakeman	• Added project structure description • Added select patient req. • Removed hash function injection from diagram. • Clarified when EnableMove is high

Glossary

***ADT* – Abstract Data Type**: A data type defined by its behaviour, possible values and operations usable through an interface. 7

I²C: Inter-Integrated Circuit. A synchronous, multi-controller/multi-target, single-ended, serial communication bus. https://en.wikipedia.org/wiki/I²C 4, 19

***LDR* – Light Dependent Resistor**: An electrical component that changes its resistance depending on the amount of light that falls on it. 4, 13

***PC* – Personal Computer**: A full computer for individual use such as a laptop or desktop computer. 4

***SAN* – SynchronizationAreaNetwork**: A bus which synchronizes actions across devices. 3, 4

***UART* – Universal Asynchronous Receiver Transmitter**: A simple serial two-wire electrical communications protocol. 19

1. Introduction

Medical X-ray imaging is an essential device in modern medicine allowing doctors to detect abnormalities including bone fractures, pneumonia, tumors and foreign objects.

Modern X-ray devices display images directly on a computer monitor. This is achieved with several components including:

- XrayGenerator: generates X-ray beams. The beam is shaped into a square by a collimator
- ImageDetector: the amount of X-ray passed through the patient is detected, and translated into an image
- Geometry: positions the X-ray generator-detector relative to the patient. It may also move during the acquiring of the image(s)
- ImageProcessor: processes the image acquired by the detector
- DisplayComponent: puts the processed image on screen
- CentralAcquisition: coordinates the process of acquiring images.
  Because some actions require real-time behaviour, part of that coordination is done via a set of wires, the so-called SynchronizationAreaNetwork (SAN) bus.
  This SAN bus is nothing more than a couple of copper wires, where each wire has a specific function. All components that are involved in acquisition are connected to that bus and can read and change the state of the wire can be at high or low voltage
- GeometryImagingModule, buttons and joysticks that the doctor can use to control the system
- PatientAdminstration components, keeps for instance track of the examinations done for a patient, the amount of dose a patient received, …

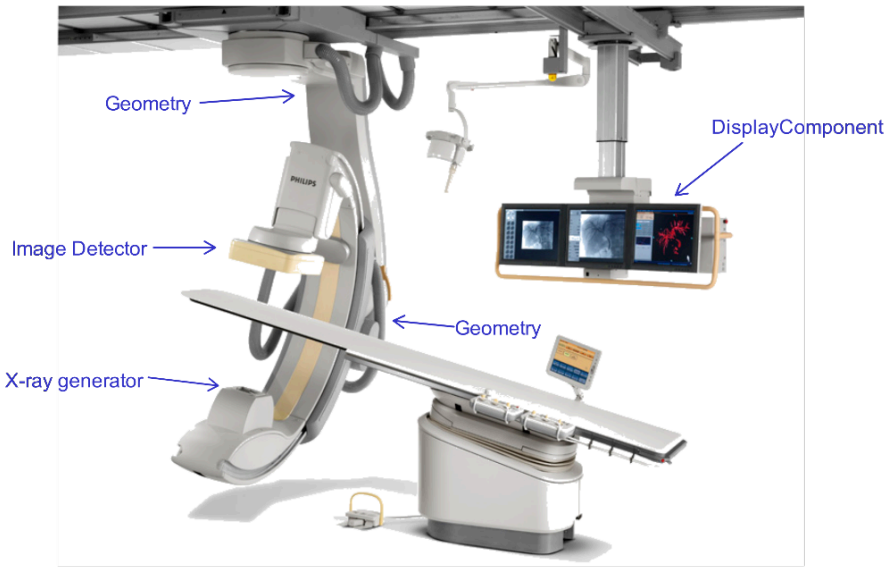


Figure 1: Components of the X-ray device

The actual behavior of a components depends on settings. The values of those settings depend on the type of examination the doctor wants to do.

As you can image an actual medical x-ray system is far more complex than described above (million lines of code, more than hundred man-years to develop, …). To simplify this project, we restrict the system to the following:

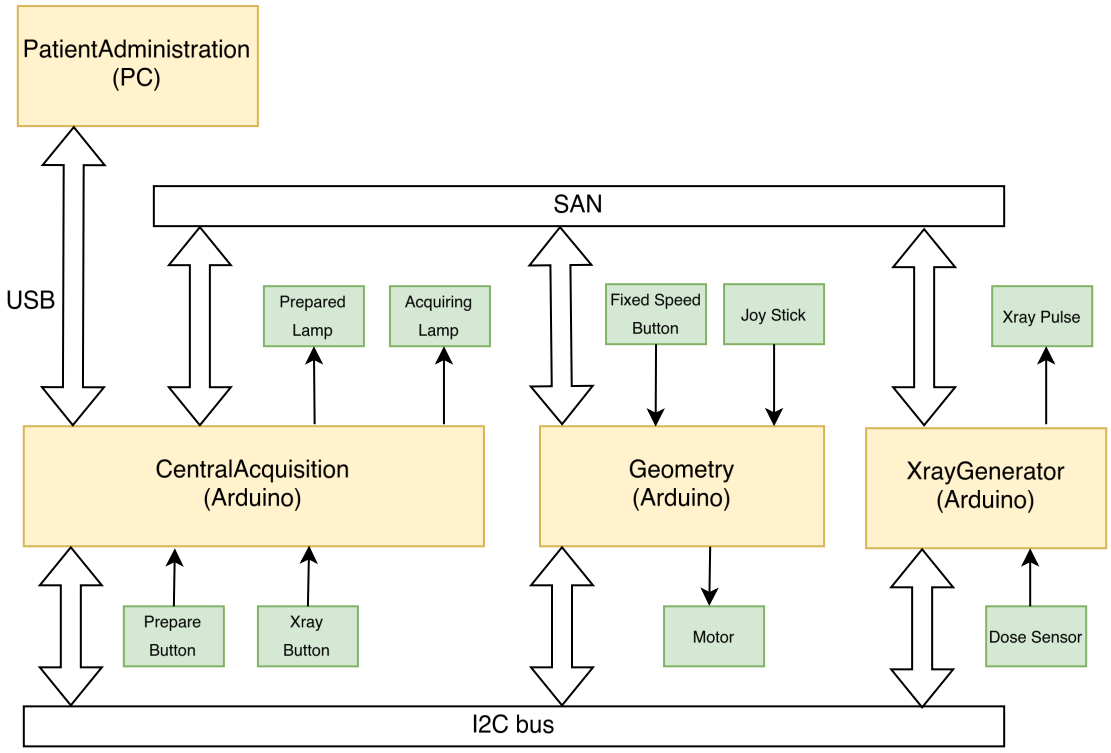


Figure 2: Block diagram of xray system

As can be seen from the diagram CentralAcquisition, Geometry and XrayGenerator act as Arduino I^2C devices, where CentralAcquisition acts as master. CentralAcquisition has a serial connection with a PatientAdministration running on a Personal Computer (PC).

From the diagram we also see that the Arduino's are also connected with each other via SynchronizationAreaNetwork (SAN). SAN is just a set of wires that can set high/low and are used to implement the real-time behavior of the system. Each wire has a specific function.

From the diagram we see that actuators and sensors are connected to the Arduinos. For most of them it is clear which actual device must be used. But for the Motor a DC or Servo motor must be used, and for the Dose sensor an Light Dependent Resistor (LDR).

2. Project structure

This project is structured in 4 sprints. Each sprint you will deliver new and improved parts of the system. In Table 1 below you can see the major sprint goals. See Section 10 for more details.

Sprint	Goals
Sprint 1	• Get environment setup • Get PatientAdministration running • Get CentralAcquisition running • Get communication between PC and Central Aquisition working
Sprint 2	• Improved patient administration • Get Geometry and XrayGenerator communicating • Improve CentralAcquisition
Sprint 3	• Complete doseAdmin API without storage. • Implement direct hardware access • All arduino components complete
Sprint 4	• Complete saving/loading of doseAdmin (patient records) • Support infinate number of patients • Replace USB/UART connection with TCP connection • Final demo

Table 1: General project structure

3. PatientAdministration Requirements

Patient administration is, among other things, responsible for scheduling examinations and keeping track of the examinations that are performed on a patient. In our case we only keep track of patients that are examined or will be examined and the X-ray dose they received during an examination. Below a list of requirements that must be implemented is given.

Category	Num	Description
PATADMIN	1	At startup the following actions are performed (in the given order): • A patient dose administration is created. If an administration is stored in persistent memory (i.e. file) then the administration is filled with that stored data. Otherwise, an administration is created that only contains 1 patient, named JohnDoe (JohnDoe is always present in the administration). • PatientDoseAdministration tries, only once, to connect with CentralAcquisition. On success it transits from the not connected state to the connected state. • Selected examination type is set to `EXAM_TYPE_NONE` • Selected patient is set to JohnDoe Rationale for JohnDoe: in case of an emergency patient of which the credentials are unknown, still a patient needs to be selected. In such a case JohnDoe is selected as patient and must therefor always be present.
PATADMIN	2	A patient can always be set as the selected patient, unselecting a patient is not possible.
PATADMIN	3	At shutdown the following actions are performed (in the given order) before ending the application: a. If a patient named JohnDoe is stored in the administration then that patient is removed. b. If the patient admin still contains one or more patients, then all patient data is stored in persistent memory (i.e. file). c. If PatientDoseAdministration is connected to CentralAcquisition then that connection is broken down.
PATADMIN	4	It is always possible to add a patient in the administration. However, the name of the patient must be unique, is at most 80 characters long and does not equal JohnDoe.
PATADMIN	5	It is not possible to delete a patient when patient to be deleted equals the selected patient or if the patient is JohnDoe, otherwise it is always possible.

PATADMIN	6	Selection of an examination type is only possible when the system is connected to CentralAcquisition and CentralAcquisition was not acquiring images.
		The following examination types can be selected:
		`EXAM_TYPE_SINGLE_SHOT`
		`EXAM_TYPE_SERIES`
		`EXAM_TYPE_SERIES_WITH_MOTION`
		`EXAM_TYPE_FLUORO`
		`EXAM_TYPE_NONE`

PATADMIN	7	When dose data is received it will be added to the selected patient. After that, the selected examination type is set to `EXAM_TYPE_NONE` (and passed to CentralAcquisition).

Category	Num	Description
PATADMIN	8	PatientAdministration runs on a linux platform, and the administration needs to be implemented as an Abstract Data Type (ADT) as described in this document
PATADMIN	9	The UI of PatientAdministration application is a menu-based console application

Table 2: Patient administration requirements

4. CentralAcquisition Requirements

4.1. Hardware requirements

The central acquisition device has two status LEDs connected to it. A red LED indicates whether an examination type is set, and a green LED indicates whether examination type is set to `NO_EXAM`.

You must connect the LEDs using a resistor and show by means of calculations what resistor is needed for the operating voltage and current of the LEDs.

Communication between the central acquisition system is through UART (over USB).

You design the protocol for communication. You can assume your protocol is connection oriented. Furthermore, you have to make sure the protocol is independent of the physical layer (i.e. if UART is replaced by sockets your protocol still works.)

4.2. Software Requirements

Central acquisition coordinates the process of acquiring of images. Below a list of requirements that must be implemented is given.

Category	Num	Description
`CentralAcq`	1	At startup CentralAcquisition is in the `NotConnected` state and waits only for the connect message, examination type is `EXAM_TYPE_NONE`
`CentralAcq`	2	When `Connect` message is received it enters the `Idle` substate of the `Connected` state
`CentralAcq`	3	Only in the `Idle` substate the reception of a select examination type is handled, and an acknowledge is replied. When not in the `Idle` state a not acknowledge is replied. On success, the received examination type is passed to Geometry and XrayGenerator
`CentralAcq`	4	When in the `Idle` substate the Prepare button changes to pressed and the examination type is not equal to `EXAM_TYPE_NONE`, it requests Geometry and XrayGenerator to prepare for acquisition and enters the preparing substate.
`CentralAcq`	5	When in the preparing substate it checks if both Geometry and XrayGenerator are in the prepared state, when that is the case, it enters the prepared substate and switches on the Prepared Lamp
`CentralAcq`	6	When in the preparing substate for more than 1 second, it sends an unprepare message to Geometry and XrayGenerator and enters the Idle substate
`CentralAcq`	7	When in the preparing substate and the Prepare button changes to not pressed, it sends an unprepare message to Geometry and XrayGenerator and enters the Idle substate
`CentralAcq`	8	When in the prepared substate the X-ray button changes to pressed the Acquiring substate is entered and `SAN_XRAY_ENABLED` is set high and remains high as long as the X-ray button is pressed. The Prepared Lamp is switched off and the Acquiring Lamp on
`CentralAcq`	9	When in the acquiring substate and the X-ray button changes to not pressed it performs the following actions: • switch off the Acquiring Lamp • `SAN_XRAY_ENABLED` is set low

Category	Num	Description
		• set the examination type to `EXAM_TYPE_NONE` and passes that type to Geometry and XrayGenerator • retrieves the received dose from XrayGenerator and passes it to PatientAdministration • enter the Idle substate
`CentralAcq`	10	When in any of the Connected substates the Disconnect message is received it performs the following actions: • switch off all lamps • `SAN_XRAY_ENABLED` is set low • set the examination type to `EXAM_TYPE_NONE` • enter the NotConnected state

Table 3: Central Acquisition Requirements

5. Geometry Requirements

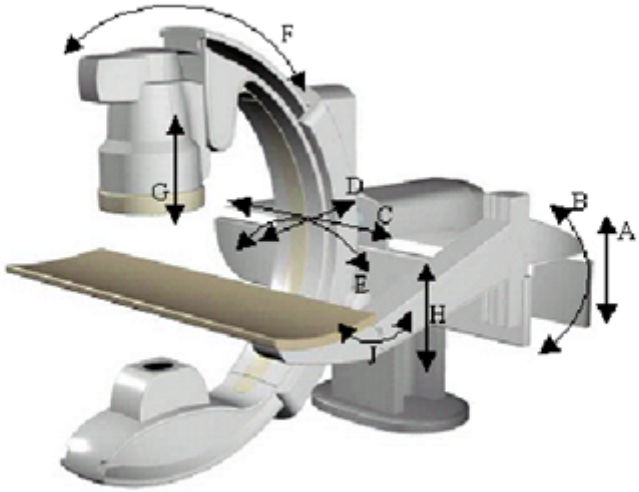


Figure 3: X-ray system motion axis

Geometry is responsible for the positioning of the detector and xray source around the patient. In an real X-ray system up to 10 electro motors can be present to do that job. All off them may run at the same time. Some motors are equipped with a brake, others not because its corresponding movement is self-locking. The doctor can position the detector X-ray source combination around the patient via joysticks and buttons. When no joystick or button is activated, motion is not possible (dead-man control).

In our case we stick to only one motor that can be controlled via a button and joystick. Below a list of requirements that must be implemented is given.

Category	Num	Description
GEO	1	At startup Geometry is in the Idle state.
GEO	2	Motion is possible in any Geometry state.
GEO	3	The motor will rotate at the half of the maximum speed if the FixedSpeed button is pressed.
GEO	4	The motor will rotate at a speed depending on the position of the joystick speed if the joystick is not in the zero position.
GEO	5	When the FixedSpeed button is activated before the joystick, the joystick is ignored if the FixedSpeed is activated. The same for the other way around
GEO	6	When prepare is received, it waits some time before going to the prepared state. The wait time mimics the prepare actions that need to be done. The wait time is a random value between [0.1, 0.4] seconds. But when the selected examination type is `SERIES_WITH_MOTION` it is between [0.3, 0.8] seconds. On reception of the prepare, geometry enters the Preparing state. After the wait time Geometry enters the Prepared state.
GEO	7	When in the preparing or prepared state and unprepare is received, it waits some time before going to the Idle state. The wait time mimics the unprepare actions that need to be done. The wait time is a random value between [0.0, 0.2] seconds.

Category	Num	Description
GEO	8	When examination type equals `EXAM_TYPE_SERIES_WITH_MOTION` and the state is Prepared and `SAN_XRAY_ENABLED` is High and EnableMove changes from Low to High then and only then the state becomes acquiring and `SAN_GEO_MOVING` is set High, and remains High only as long as EnableMove is High and `SAN_XRAY_ENABLED` is High. EnableMove is high if and only if the FixedSpeed button or joystick is activated. When `SAN_GEO_MOVING` is changed from High to Low, transits from Acquiring to the Idle state.
GEO	9	Geometry has registers that can be read and written by CentralAcquisition. The registers contain information about the selected examination type, state of Geometry and the hours, minutes and seconds the motor was running since uptime. But also, a register in which a command (e.g. prepare) can be written. Addresses of the registers and their definition: Register $8 7 6 5 4 3 2 1 0 x x cmd x x x x Command *01 = Prepare* *10 = Unprepare* Register $9 7 6 5 4 3 2 1 0 et x x gs x XrayGenerator state *00 = Idle* *01 = Preparing* *10 = Prepared* *11 = Acquiring* Examination Type *000 = EXAM_TYPE_SINGLE_SHOT* *001 = EXAM_TYPE_SERIES* *010 = EXAM_TYPE_SERIES_WITH_MOTION* *011 = EXAM_TYPE_FLUORO* *100 = EXAM_TYPE_NONE*

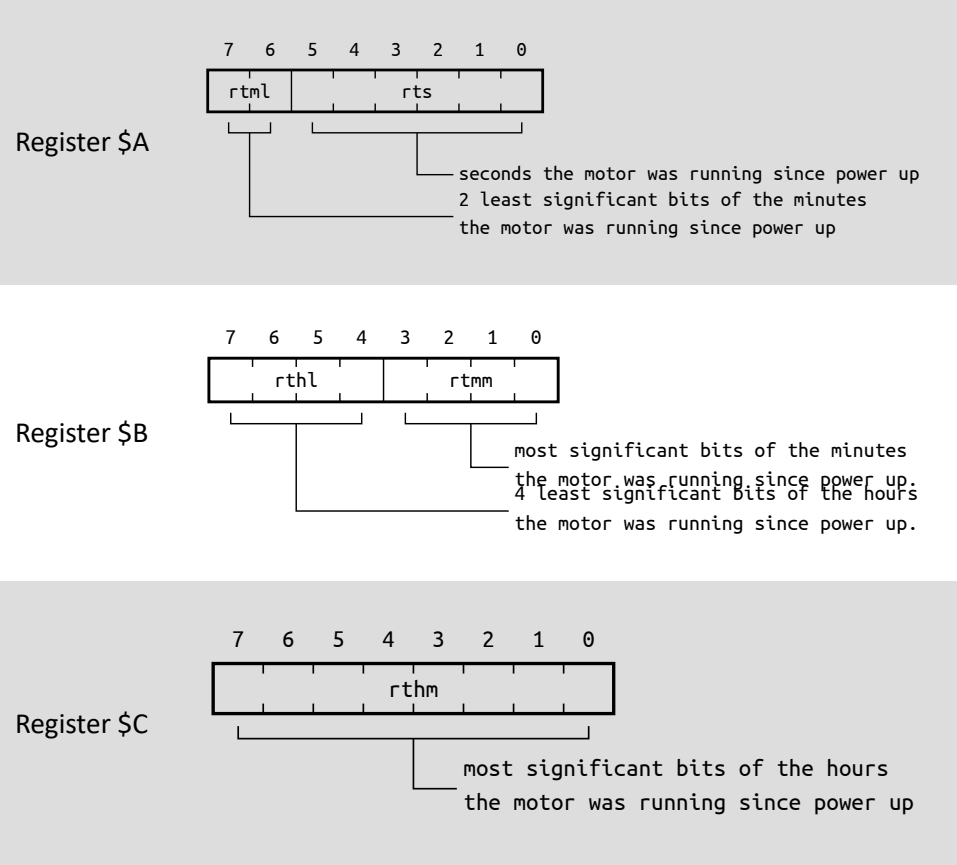
Category	Num	Description
		 At startup register $9 has value `0x80`, and the registers $A … $C and $8 value `0x00`.

Table 4: Geometry requirements

6. X-ray Generator Requirements

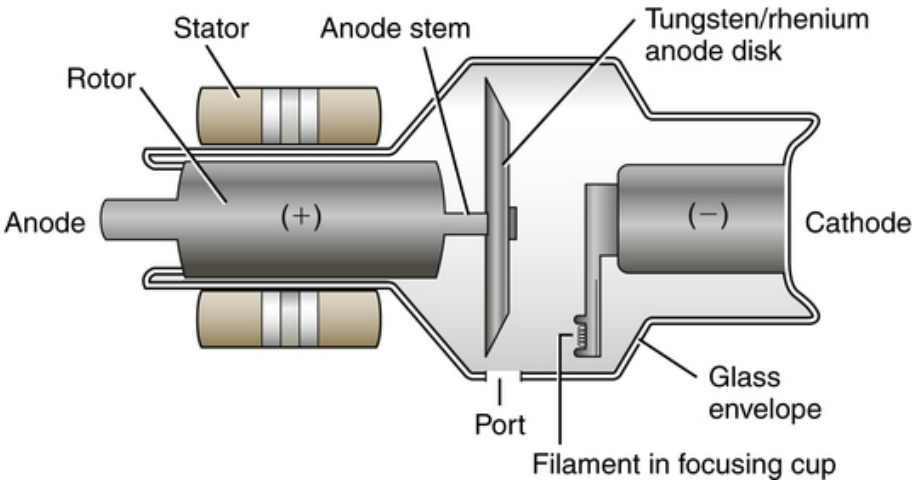


Figure 4: X-ray generator cross cut

XrayGenerator generates the X-ray beam. This is basically done by creating a very high voltage difference between the anode and cathode. Due to the high voltage potential, electrons escape from the cathode and bombard the rotating anode disk, generating X-rays. The rotating of the disk is for cooling purposes and to extend the lifetime of the tube. The X-rays can escape from the glass/lead envelope via a small port in the tube. Next, the escaped X-rays are filtered to remove X-rays that will never reach the detector because they are too weak. Furthermore, the escaped X-ray is shaped by the collimator in such a way that only the detector will be exposed to X-rays. Finally, the amount of X-rays that are emitted towards the patient is measured, or calculated by some mathematical model. Remark that the filter, collimator and dose measurement devices are not depicted.

In our case the actual generation of a X-ray pulse is mimicked by a LED and a LDR is used for dose measurement.

6.1. Hardware requirements

Attached to the X-ray generator is an X-ray emitter. This emitter is highly sensitive and requires an exact current to operature within safe limits. The current through the device must be above 15mA, but never exceed 19mA. If the current is too low, the dose is too low resulting in an unclear picture. If the current is too high, the dose is too high and is dangerous for humans. The optimal current for the device is 17mA.

Note: the X-ray emitter is simulated with an LED.

X-ray emitter

You must connect the LED to the device and make sure that the current through the LED is within specifications.

For safety reasons you must do the following:

1. Proof that the amount of current through the LED is within specification using Ohm's law calculations.
2. Measure the current through the LED and take a picture of the multimeter.

> **Caution**
>
> A multimeter can be damaged if you are measuring current. Watch this https://www.youtube.com/watch?v=EvAq9zqRB5I video tutorial for a good introduction or ask teacher for assistance if you are unsure.

Is there is a discrepancy between the calculated value and the actual measurement? Why is that? How large is the error in percentages?

Draw a circuit diagram of the arduino, and all the components.

X-ray measurement sensor

For additional safety and calibration reasons the emitted X-rays must be measured with an X-ray sensor so that the dose received by the patient is known.

Note: the X-ray sensor is simulated with an LDR.

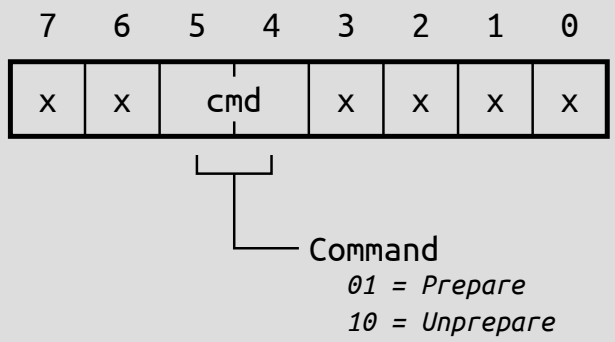
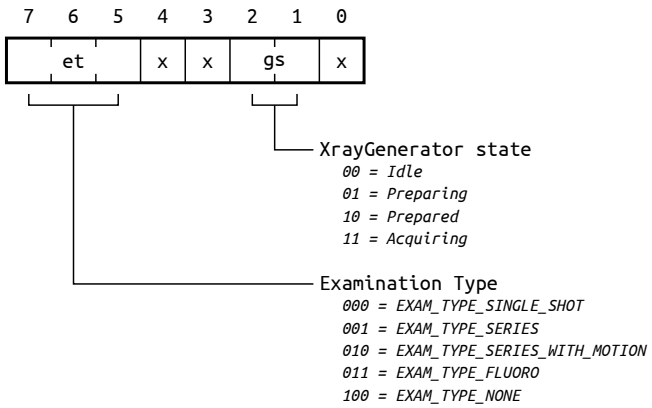
To connect the LDR to the system a voltage divider is used. It is up to you to design the divider.

1. Show calculations (Ohm's law / Kirchoff's laws) that proof that the LDR uses the full range(0-5V) from darkness to maximum LED brightness at 23mAh[1].
2. Measure the voltages at the divider and take a picture of the multimeter showing the different voltages. Measure the voltages for darkness, brightness and at 15mAh, 17mAh and 19mAh.
3. Draw a diagram that shows the voltage divider and the LDR.

6.2. Software requirements

Category	Num	Description
XRAY	1	At startup XrayGenerator is in the Idle state.
XRAY	2	Generating X-ray is impossible when `SAN_XRAY_ENABLED` is Low or examination type is `EXAM_TYPE_NONE`.
XRAY	3	When in the idle state and prepare is received, it enters the preparing state. In the preparing state it sets the register that will contain the received dose to zero, it prepares all that is needed for the timing of the xray pulse(s). To mimic the unpredictable timing behavior of the prepare actions a random wait time between [0.1,w 0.2] seconds is added. After all preparing actions are done it enters the Prepared state.
XRAY	4	When in the preparing or prepared state and unprepare is received, it waits some time before going to the Idle state. The wait time mimics the unprepare actions that need to be done. The wait time is a random value between [0.2, 0.4] seconds.
XRAY	5	When in the preparing or prepared state and unprepare is received, it waits some time before going to the Idle state. The wait time mimics the unprepare actions that need to be done. The wait time is a random value between [0.2, 0.4] seconds.
XRAY	6	When in the prepared state and selected examination type is `EXAM_TYPE_SERIES_WITH_MOTION` and SAN_GEO_MOVING is High and `SAN_XRAY_ACTIVE` changes from Low to High the Acquiring state is entered.
XRAY	7	When in the Acquiring state and examination type is EXAM_TYPE_SINGLE_SHOT then one X-ray pulse is generated. The pulse has a duration of 10 milliseconds. The power of the pulse is at its maximum level.
XRAY	8	When in the Acquiring state and examination type is `TYPE_SERIES` or `EXAM_TYPE_SERIES_WITH_MOTION` then a series of X-ray pulses is generated. Each

[1]a rule of thumb is to use 10%-20% headroom. Since the maximum LED ouput is 19mAh: $19 \times 1.20 \approx 23\mathrm{mAh}$)

Category	Num	Description
		pulse has a duration of 10 milliseconds. The time between two pulses is fixed, 0.49 seconds. The power of the pulses is at its maximum level.
XRAY	9	When in the Acquiring state and examination type is `EXAM_TYPE_FLUORO` then a series of X-ray pulses is generated. Each pulse has a duration of 10 milliseconds. The time between two pulses is fixed, 0.24 seconds. The power of the pulses is at a 10th of the maximum level
XRAY	10	When in the Acquiring state and `SAN_XRAY_ENABLED` or `SAN_GEO_MOVING` changes from High to Low then the Idle state is entered
XRAY	11	When during a pulse `SAN_XRAY_ENABLED` or `SAN_GEO_MOVING` changes from High to Low then the pulse is completed before going to the Idle state.
XRAY	12	During a pulse the dose is measured as often as possible. The moving average of those measurements is added to the content of the register that contains the received dose. The size of the moving average filter is 5.
XRAY	13	XrayGenerator has registers that can be read and written by CentralAcquisition. The registers contain information about the selected examination type, actual state of XrayGenerator, the number of pulses at max power generated since uptime and the number of pulses at 10th max power since uptime. But also, a register in which a command (e.g. unprepare) can be written. Addresses of the registers and their bit definition: Register $8  Register $9 

Category	Num	Description
		Register $A…$D — Contains the number (`uint32_t`) of pulses at the maximum power since power up. $A contains the most significant byte of that number, $D the least significant byte. Register $E …$11 — contains the number (`uint32_t`) of pulses at the 10th of the maximum power since power up. $E contains the most significant byte of that number, $11 the least significant byte. At startup register $9 has value `0x80`, and the registers $A … $11 and $8 value `0x00`.

Table 5: X-ray Generator Requirements

7. Patient Administration ADT Details

A hash table is used to store patient dose data in such a way that data can be found and processed faster. During the sprint 3 different implementations of the administration need to be made (in different sprints). But remark that the API for all implementations is the same. See section Sprint Content for details about these different implementations. How the linked list implementation looks like is shown below.

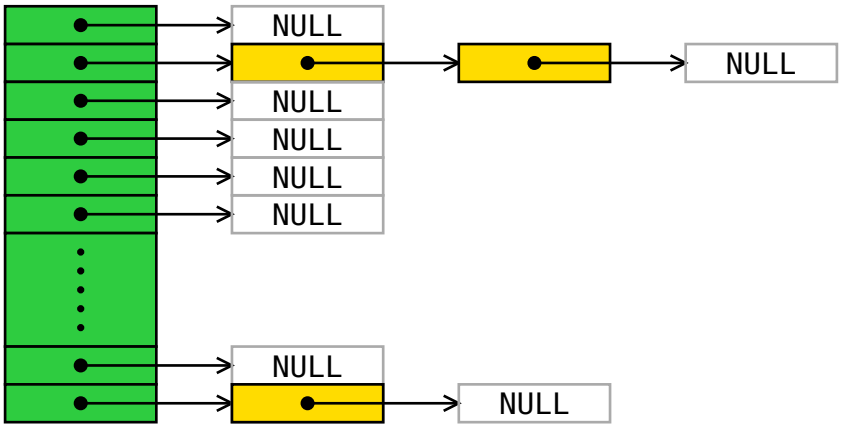
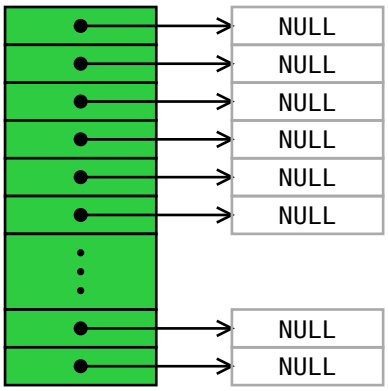


Figure 5: Example implementation of a linked list.



The hash table, is an array of size 256. The entries of this array contain a pointer pointing to a linked list, and the number of patients present in the hash entry. At startup all entries have a pointer equal to `NULL`, and number of patients equal to 0.

Which index of the array has to be used is calculated by the so-called **hash function**. A possible implementation of that hash function could be: add the value of the first 5 characters of the patient name and take the modulo 256 of that sum. This results in a value in the range 0..255. That value is the index in the hash table array for storing patient data. When patientname has less than 5 characters then character value 255 for the missing characters is used in the summing. The hash function may differ for each created table.

Note that this array is almost the same for both implementations. The array only implementation points to an array, instead of pointing to a linked list.

An element of the linked list containing patient data, it has the following members:

- **name** of the patient a `\0` terminated character array, the max size of the name is 80 characters. So the array size has to be 81.
- **dose** data. Is an array of DoseData structs, size of array is 10 DoseData struct has the folowing members:
  - the amount of **dose** received in an examination
  - the **date** of the examination, is a string in the format `dd-mm-yyyy`
- **number of measurements**. The number of measurents done for the patient.
- **next**, pointer to the next element

Listing 1: Legend

8. API

See file `GivenStartingCode/Shared/doseAdmin.h` for a detailed description of the API.

Some implementation hints/remarks:

- Remark that the API does not expose the internal structure of the implementation, that is what it makes it an abstract data type.
  But also remark that the test code must not depend on the internal structure of the implementation.
- In case of the array only implementation, the needed arrays may have a fixed size of 100 at startup. When during runtime the array should become bigger (e.g. patient 101 needs to be added in a hash entry) then a new array of for instance size 200 could be allocated.

9. Protocol requirements

The individual components communicate together using different protocols. The components inside the X-ray device communicate with each other using I^2C. The X-ray device communicates with the display component using Universal Asynchronous Receiver Transmitter (UART).

Design a message protocol for the communication between the devices.

10. Sprint Content

You need to make document that describes your product. That document must have a front page, table of content, revision history and introduction. The same document is used for each sprint. Remark that work (e.g. diagrams) done in a previous sprint need to be updated in the running sprint, this task will not be mentioned explicitly in the sprint tasks given below.

The first sprint is individual, but after that sprint the teacher can decide to make groups of 2 students.

At the end of each sprint all code and the document need to be put in Canvas (only a link to your Git repository is not ok).

General implementation constraints:

- Code must adhere to the coding standards
- It is not allowed to use delay in code running on Arduino
- Debouncing of buttons must be fixed via software, not hardware
- PatientAdministration must run on a Linux platform

10.1. Sprint 1

Sprint1 Content/Tasks	**Needed Knowledge**
In case of Windows, setup WSL	Is described in Canvas course
Setup GIT project, and give access to teachers	GIT workshop
Copy the given project in your GIT and get it up and running. To get it up and running you need to attach an Arduino (Uno) to your laptop. The Arduino (CentralAcquisition) is also given. Read the ReadMe file and follow the instructions!	Make
Implement the following functions of the doseAdmin API, and if applicable use them in the console menu • Implement the hash function (see `doseAdmin.c`) • CreateHashTable, using only arrays. You may assume that a HashTable entry can contain at most 100 patients, and that a patient has at most 10 dose measurements. • RemoveAllDataFromHashTable • AddPatient • RemovePatient • IsPatientPresent	Arrays, structs, strings, functions
Adapt the code (laptop and Arduino) so that an examination type can be set on CentralAcquisition. To visualize this set a LED when an examination type is set, set another LED on/off to indicate whether examination type `NO_EXAM` is selected or not. Do a resistor current/voltage calculation for the LED's. Document the protocol between PC and CentralAcquisition.	LED, pin mode, digital write, electric basics (resistor), LED data sheet, Falstad, Multimeter
Make a context diagram of the console application	Context diagram
Make a use case diagram and use case descriptions for the console application	Use case diagram notation, use case description notation
Make a module structure diagram of the console application.	Module structure notation

10.2. Sprint 2

Sprint2 Content/Tasks	Needed Knowledge
Implement all functions of doseAdmin API (except the read from and write to file functions). Now a hash entry has at most 1 patient. But the memory for that patient is now dynamically allocated. Still a patient has an array member of size 10 for storing dose data. Memory leaks are acceptable for now.	Pointers,malloc, free Stack vs Heap (python tutor) Algorithm basics (average, standard deviation)
Make testcases for all implemented doseAdmin API functions.	Testcases, Unity
Make a context diagram of CentralAcquisition	
Draw and implement the state diagram running on the pc	State diagrams
Draw and implement the state diagram running on CentralAcquisition	
Make a use case diagram and use case descriptions for CentralAcquisition	
Make a module structure diagram for CentralAcquisition	
Connect via I²C Geometry to CentralAcquisition. Also, on Geometry and XrayGenerator, visualize via a LED when an examination type is set, set another LED on/off to indicate whether examination type `NO_EXAM` is selected or not	I²C,Arduino Wire lib
Verify your own protocol using logic analyzer sniffing the uart connection	Logic analyzer

10.3. Sprint 3

Sprint3 Content	Needed Knowledge
Implement the doseAdmin API (except the read from and write to file functions) with linked list, so now a hash entry can have an arbitrary number of patients. Still, a patient has an array of max 10 dose measurements.	Linked list, valgrind, memory usage on a mcu
Implement pin mode via registers, timers and interrupts needed in this sprint as well	Baremetal programming on Arduino Uno, Atmel328 datasheet, bit manipulation
Make a context diagram for Geometry and XrayGenerator	
Describe the communication protocol between CentralAcquisition and Geometry and XrayGenerator. Also make headerfiles that address the registers of Geometry and XrayGenerator.	
Make a UseCase diagram and UseCase description for Geometry and XrayGenerator	
Make a state diagram for Geometry and XrayGenerator	
Make a complete implementation of all Arduino components, including all lamps, motor, buttons,…	Relevant datasheets
Make screenshot of the logical analyzer display that show the correct generation of the xray pulses (for SingleShot and Series)	Logic analyzer

10.4. Sprint 4

Sprint4 Content	Needed Knowledge
Implement the read from and write to file functions of the doseAdmin API. Add testcases.	File IO
In the previous sprint max 10 dose measurements can be stored. In this sprint that array of size 10 has to be replaced by a linked list.	
Replace the USB connection by sockets	Network basics, sockets, wireshark
Implement your backlog, when this is the case	
Rework given feedback, if not done yet	
Finalize your document and code	
Give final demo	